

Uncertainty Signals for Handwritten Math: Initial Results

Qwen2.5-VL-3B on FERMAT

The question

Can a model tell us when it is wrong, without seeing the answer key? We tested one idea. Ask the model the same question five times, then measure how much its five answers disagree with each other (Shannon entropy). If the model disagrees with itself more on the items it gets wrong, that gives us a free way to catch errors in real use, where nobody knows the right answer yet.

We tested this on two tasks: reading a handwritten math solution, and judging whether that solution contains a mistake.

Setup

Qwen2.5-VL-3B on FERMAT. Five samples per prompt at temperature 0.7, plus two greedy samples to check the sampling code works. Entropy is computed only from the model’s own five answers; correctness is scored separately against ground truth. AUROC tells us whether the two line up: it is how often a randomly picked wrong item had more disagreement than a randomly picked correct one, so 0.5 means no relationship. Ranges are 95% bootstrap confidence intervals. We ran 300 items, half containing a mistake and half not, with thresholds fixed in advance.

Results

Task	AUROC (95% CI)	Model accuracy	Verdict
Reading handwriting	0.835 [0.787, 0.879]	0.393	entropy predicts error
Judging mistakes	0.522 [0.462, 0.582]	0.517 (chance = 0.500)	no signal

Reading handwriting: it works (works)

When the model cannot read the handwriting it genuinely wavers, and that wavering lines up with its mistakes. The result holds under every check we ran. Throwing away every item where parsing failed and every item with maximum disagreement still leaves AUROC 0.796 [0.736, 0.852] on the remaining 206 items, and it matches an earlier 100 item pilot that gave 0.788.

A simple rule comes out of this. **When all five readings differ, the model is wrong 57 times out of 61, or 93.4%.** That catches 31% of all reading errors, and you can apply it without knowing the right answer. Two real items:

Five identical readings, and correct

correct answer	option d
5 readings	option d, option d, option d, option d, option d

Five different readings, and wrong

correct answer	$A - B \neq B - A$
5 readings	$A - B = \{1, 3, 5\}$ and $B - A = \{8\}$ — $A = B \neq B - A$ — $A = B \neq B = A$ — $A \neq B$ — $A \neq B \neq B - A$

The second is genuinely messy handwriting. The model cannot settle on a reading and every attempt is different. That instability is the signal.

Only 3 of the 300 items had all five readings agreeing and were still scored wrong, and all three are scoring quirks rather than real mistakes, for example the model answering **option a** where the correct

answer was written `\text{A}`. So the 0.393 accuracy is a strict lower bound, not our best estimate: ignoring formatting gives 0.527, and accepting the model’s answer when it appears inside the correct line gives 0.640.

Importantly, **the AUROC barely moves across all of these**: 0.835 strict, 0.785 ignoring formatting, 0.777 under the loosest rule, and 0.814 when the whole pipeline is re-run with a different L^AT_EX parser. The signal does not depend on how strictly we score correctness.

Judging mistakes: no signal, because the model cannot do the task (fails)

The reason matters more than the number. In FERMAT, 84.8% of solutions contain a mistake. On a sample with that same imbalance the model scored 75%, which sounds reasonable until you notice that always answering “yes there is a mistake”, without looking at anything, scores 87%. The model was doing worse than that. On a balanced set the flattery disappears:

Accuracy on the balanced set	0.517	(chance = 0.500)
on solutions that do contain a mistake		0.947
on solutions that are correct		0.087
Answers “there is a mistake”		93% of all items

The model has one firm opinion, that every solution contains a mistake, and it repeats it. So it almost never disagrees with itself: 42% of items had all five answers identical, and only three different entropy values appeared in the whole run. There is nothing for entropy to measure.

What this means

Entropy measures whether a model is consistent with itself. It never looks at whether the model is right. The method only works when those two things go together, meaning the model is steady when it is right and wavers when it is wrong. That holds for reading handwriting. It fails for judging mistakes, because a model that is confidently wrong looks exactly like one that is confidently right. Both look steady.

The clearest evidence is a smaller result. On the solutions that were actually correct, AUROC is **0.239**, which is below 0.5. That means entropy ran backwards. The model was at its most consistent exactly when it was wrong, insisting on mistakes that were not there. Low disagreement there is actively misleading, not just uninformative.

So the headline is not that entropy fails for reasoning. It is that **this kind of uncertainty inherits the model’s ability**. It can flag errors for a model that knows something, and it tells you nothing about a model that is guessing. We only found this by rebalancing the test set. On the original imbalanced data the problem was invisible.

Next step

The judging result is about this model, not the method. The obvious test is a stronger model (7B, or a text-only model given the correct transcription), with one condition: check its accuracy against the 0.500 baseline first, before computing any uncertainty numbers. If it still scores near chance, the negative result holds for this kind of task. If it reaches roughly 0.65 to 0.70, the question becomes answerable again and our existing code applies unchanged.

The handwriting result is usable as it stands and deserves a threshold analysis: how precision and recall trade off as the cutoff moves, and conformal calibration for a guaranteed error rate. That needs no new model runs.

Full methodology, the earlier failed approaches and the 100 item pilot are in [report/report.pdf](#); code is unit tested in [pilot/](#).